

Trabajo Práctico Integral

Integrantes:

- Ducoli, Maximiliano
- Elizondo, Iñaki
- Farias, Gustavo

Comisión: M2026-13

Grupo: 132

Repositorio GitHub: (en desarrollo)

1. Identificación de problemática y propuesta de solución

1.1 Contexto claro y estado del proceso

En la actualidad, los usuarios enfrentan problemas cotidianos en sus hogares, oficinas, tecnología y educación (por ejemplo, una pérdida de agua, un enchufe que no funciona, dudas con una materia escolar).

Estado actual: El usuario busca en internet (información dispersa y no curada), pregunta en redes sociales (depende de la disponibilidad de terceros) o contrata a un profesional "a ciegas" (sin conocer su reputación real, generando desconfianza y posibles sobreprecios).

Conclusión del relevamiento: Existe una brecha entre el conocimiento técnico especializado y el usuario común, lo que genera pérdida de tiempo, dinero e inseguridad.

1.2 Análisis de la Problemática (Actores y Flujo)

- Usuario final: Necesita una respuesta rápida, ver tutoriales en video confiables y, si no puede resolverlo, contactar a alguien de confianza.
- Especialista (plomero, electricista, docente): Necesita visibilidad, clientes calificados y construir reputación sin gastar en marketing.
- Empresa (administrador): Necesita generar una plataforma útil, blindarse legalmente y crear un modelo de negocio escalable.

1.3 Propuesta de Solución Tecnológica

Desarrollar una Web App Responsive que funcione como un ecosistema de resolución de problemas, estructurada en dos niveles de entrega para garantizar la viabilidad temporal:

Nivel 1 (Core Garantizado - MVP Base): Un motor de búsqueda inteligente que conecta directamente con la API de YouTube para mostrar los 3 mejores videos tutoriales basados en la consulta del usuario, junto con un directorio de Especialistas verificados manualmente y ordenados por reputación.

Nivel 2 (Ampliación - MVP con IA): Integración de un Asistente de IA (API de Ollama) que actúe como el "cerebro" del sistema para probar la conexión y funcionamiento. La IA procesará la consulta para generar un diagnóstico paso a paso, extraer las palabras clave para la búsqueda de YouTube y clasificar la categoría para sugerir los especialistas más adecuados.

1.4 Validación de la Solución

Eficacia (Lograr el objetivo): El usuario siempre obtendrá videos tutoriales y opciones de contacto humano, independientemente de si la IA está integrada o no en esa instancia.

Eficiencia (Uso óptimo de recursos): Automatiza la curación de contenido (vía API de YouTube) y la conexión de ofertas y demandas (Marketplace), reduciendo el tiempo de búsqueda a segundos.

2. Análisis de la Problemática (Actores y Flujo)

En lo que respecta al alcance del desarrollo, como primer MVP desarrollaremos un modelo de App sin la incorporación de IA.

Con base en los avances que se vayan realizando en el desarrollo, se evaluará la posibilidad de añadir la funcionalidad de IA como primer acercamiento a las dudas y consultas del usuario.

2.1 Actores del Sistema

- Usuario registrado: Puede usar el buscador y contactar especialistas.
- Especialista: Puede postular su perfil.
- Administrador: Valida perfiles y gestiona categorías.

2.2 Casos de uso

CU-01: Consultar buscador de soluciones

- Actor principal: Usuario registrado
- Actores secundarios: API de Youtube, API de IA Ollama (opcional), Base de Datos PostgreSQL (Supabase).

Descripción:

El usuario ingresa una consulta describiendo un problema cotidiano (por ejemplo, "pileta tapada" o "enchufe que no funciona"). El sistema procesa la consulta, recupera contenido relevante de Youtube y muestra los especialistas más adecuados según la categoría detectada.

Precondiciones:

- El usuario debe haber iniciado sesión (CU-02).
- Debe existir al menos un especialista aprobado en la base de datos.
- La API de Youtube debe estar disponible.

Postcondiciones:

- El sistema muestra al usuario 3 videos tutoriales y una lista de especialistas ordenados por reputación.
- Se registra la consulta en el historial del usuario.

Flujo básico:

1. El usuario ingresa al módulo de búsqueda e introduce su consulta en lenguaje natural.
2. El sistema envía la consulta al backend.
3. (Si la IA está disponible) El backend invoca la API de Ollama para estructurar la consulta: extrae palabras clave, diagnostica el problema y determina la categoría (Plomería, Electricidad, Educación, Tecnología).
4. (Si la IA no está disponible) El backend utiliza el texto crudo del usuario como palabras clave y asigna una categoría por defecto o por coincidencia simple.
5. El backend consulta la API de Youtube con las palabras clave y obtiene los 3 videos más relevantes.
6. El backend consulta PostgreSQL (Supabase) filtrando especialistas aprobados de la categoría detectada, ordenados por calificación promedio descendente, y selecciona los 3 mejores.
7. El sistema arma la respuesta combinada (videos + especialistas) y la envía al frontend.
8. El frontend renderiza la sección de videos embebidos y la lista de tarjetas de especialistas.

Flujos alternativos:

3. La API de Youtube no responde: El sistema muestra un mensaje de error controlado e invita al usuario a intentar nuevamente o a explorar directamente el directorio de especialistas.

6. No hay especialistas aprobados en la categoría detectada: El sistema muestra únicamente los videos y un mensaje indicando que no hay especialistas disponibles para esa categoría en este momento.

CU-02: Registrarse / Iniciar sesión

- Actor principal: Usuario / Especialista.
- Actores secundarios: Proveedor de identidad de Google.

Descripción:

El usuario accede a la plataforma mediante autenticación con su cuenta de Google, sin necesidad de crear credenciales locales.

Precondiciones:

- El usuario debe poseer una cuenta de Google válida.
- La aplicación debe estar correctamente configurada en Google Cloud Console con las credenciales OAuth.

Postcondiciones:

- El usuario queda autenticado y con una sesión activa (JWT).
- Si es la primera vez que ingresa, se crea automáticamente un registro en la tabla Usuario.

Flujo básico:

1. El usuario hace clic en el botón "Iniciar sesión con Google" en la landing page.
2. El frontend redirige al flujo de consentimiento de Google.
3. El usuario selecciona su cuenta y autoriza los permisos básicos (perfil y email).
4. Google redirige al backend con un código de autorización.
5. El backend intercambia el código por un token de acceso y obtiene los datos del usuario (nombre, email, foto, ID de Google).
6. El backend verifica si el usuario ya existe en la base de datos por googleId.
7. Si no existe, crea un nuevo registro en la tabla Usuario.
8. El backend genera un JWT y lo devuelve al frontend.
9. El frontend almacena el token (cookie segura / localStorage) y redirige al dashboard principal.

Flujos alternativos:

3. El usuario cancela el consentimiento en Google: El sistema lo redirige de vuelta a la landing con un mensaje informativo y sin sesión activa.

5. El token de Google es inválido o expiró: El backend responde con error 401 y el frontend muestra un mensaje solicitando reintentar el login.

CU-03: Contactar Especialista

- Actor principal: Usuario registrado
- Actores secundarios: Ninguno

Descripción:

El usuario visualiza el perfil público de un especialista y accede a sus datos de contacto directo (por ejemplo WhatsApp) para coordinar un servicio por fuera de la plataforma.

Precondiciones:

- El usuario debe estar autenticado.
- El especialista debe tener estado "Aprobado" y sus datos de contacto visibles.

Postcondiciones:

- El sistema registra una métrica de "contacto exitoso" asociada al usuario y al especialista.
- El usuario es redirigido a WhatsApp Web o a la app de llamadas del dispositivo.

Flujo básico:

1. El usuario ingresa al perfil público de un especialista (desde el listado o desde los resultados de búsqueda).
2. El frontend solicita al backend los datos públicos del especialista.
3. El backend valida que el especialista esté aprobado y devuelve la información (nombre, oficio, descripción, zona, WhatsApp).
4. El frontend muestra el perfil completo con un botón destacado "Contactar por WhatsApp".
5. El usuario hace clic en el botón.
6. El sistema registra la métrica de contacto en la base de datos.
7. El sistema abre el enlace a WhatsApp en una nueva pestaña.

Flujos alternativos:

3. El especialista fue dado de baja o bloqueado: El sistema muestra un mensaje indicando que el perfil ya no está disponible y redirige al listado general.
5. El usuario no está autenticado: El sistema redirige al flujo de login (CU-02) y, tras autenticarse, lo devuelve al perfil del especialista.

CU-04: Postular Perfil de Especialista

- Actor principal: Especialista (Usuario registrado).
- Actores secundarios: Servicio de almacenamiento de archivos (cloud storage).

Descripción:

Un usuario registrado que desea ofrecerse como especialista completa un formulario con sus datos profesionales y sube documentación para ser validado por un administrador.

Precondiciones:

- El usuario debe estar autenticado.
- El usuario no debe tener ya un perfil de especialista en estado "Pendiente" o "Aprobado".

Postcondiciones:

- Se crea un registro en la tabla Especialista con estado "Pendiente".
- El administrador recibe una notificación (email o alerta en panel) de que hay un nuevo perfil para revisar.

Flujo básico:

1. El usuario accede a la sección "Quiero formar parte".
2. El sistema muestra el formulario con los campos: oficio, categoría, descripción, zona de trabajo, teléfono de contacto, foto de DNI (frente y dorso), foto de certificado o matrícula.
3. El usuario completa todos los campos obligatorios y sube las imágenes.
4. El frontend valida que los campos estén completos y envía los datos al backend.

5. El backend valida que los campos obligatorios estén completos, sean válidos y que los archivos sean imágenes válidas.
6. El backend sube las imágenes al servicio de almacenamiento y obtiene las URLs.
7. El backend crea el registro en la tabla Especialista con estado "Pendiente" y marca al usuario como "ya tiene perfil".
8. El sistema muestra un mensaje de confirmación: "Tu perfil fue enviado y será revisado en las próximas 48 hs".

Flujos alternativos:

5. Campos o imágenes inválidas: El backend responde con errores de validación y el frontend los muestra junto a cada campo correspondiente.
7. El usuario ya tenía un perfil rechazado: El sistema permite reenviar la postulación sobrescribiendo los datos anteriores, manteniendo el historial de intentos.

CU-05: Validar Especialista

- Actor principal: Administrador.
- Actores secundarios: Servicio de comunicación.

Descripción:

El administrador revisa la documentación cargada por un especialista postulante y decide aprobarla o rechazarla.

Precondiciones:

- El administrador debe estar autenticado con rol "Admin".
- Debe existir al menos un especialista en estado "Pendiente".

Postcondiciones:

- El especialista queda en estado "Aprobado" (y es visible públicamente) o "Rechazado".
- El especialista recibe una notificación en la plataforma o por WhatsApp con la decisión.

Flujo básico:

1. El administrador ingresa al panel /admin.
2. El sistema muestra una tabla con todos los especialistas en estado "Pendiente", incluyendo nombre, oficio, zona, y enlaces a las imágenes de DNI y certificado.
3. El administrador selecciona un especialista y revisa la documentación.
4. El administrador hace clic en "Aprobar" o "Rechazar".
5. Si aprueba: el sistema cambia el estado a "Aprobado", registra la fecha y el ID del admin que aprobó.
6. Si rechaza: el sistema solicita un motivo, cambia el estado a "Rechazado" y guarda el motivo.

7. El sistema envía una notificación transaccional al especialista informando la decisión (con el motivo en caso de rechazo).
8. El especialista desaparece de la lista de pendientes.

Flujos alternativos:

2. No hay especialistas pendientes: El sistema muestra un mensaje "No hay perfiles pendientes de revisión".
6. El servicio de comunicación no responde: El sistema registra la incidencia en un log interno, pero igual completa el cambio de estado. El administrador puede reenviar la comunicación manualmente desde el panel.

CU-06: Calificar Especialista

- Actor principal: Usuario registrado.
- Actores secundarios: Ninguno.

Descripción:

Un usuario que previamente contactó a un especialista puede dejar una calificación (1 a 5 estrellas) y un comentario opcional, que impacta en la reputación pública del especialista.

Precondiciones:

- El usuario debe estar autenticado.
- El usuario debe haber registrado al menos un contacto exitoso con ese especialista (métrica del CU-03).
- El usuario no debe haber calificado ya a ese especialista (restricción de unicidad).

Postcondiciones:

- Se crea un registro en la tabla Reseña.
- Se recalcula el promedio de calificación y el total de reseñas del especialista.

Flujo básico:

1. El usuario ingresa al perfil de un especialista que contactó previamente.
2. El sistema verifica que exista al menos un contacto registrado entre ese usuario y ese especialista.
3. El sistema verifica que el usuario no haya calificado ya a ese especialista.
4. Si ambas condiciones se cumplen, el frontend muestra el formulario de calificación (selector de 1 a 5 estrellas y campo de comentario opcional).
5. El usuario selecciona la cantidad de estrellas y, opcionalmente, escribe un comentario.
6. El usuario envía el formulario.
7. El backend valida las restricciones y crea el registro en Reseña.
8. El backend ejecuta un trigger (o lógica de servicio) que recalcula el promedio de estrellas del especialista en la tabla Especialista en un horario de baja concurrencia.

9. El frontend actualiza la vista mostrando modal informando que la calificación ha sido registrada.

Flujos alternativos:

2. El usuario nunca contactó a ese especialista: El sistema no muestra el formulario y, opcionalmente, un mensaje indicando que solo pueden calificar usuarios que hayan contactado al especialista.
3. El usuario ya lo calificó: El sistema muestra la reseña existente y ofrece la opción de editarla (reemplazando la anterior).

3. Elección del stack tecnológico

3.1 Condición general y costos de cambio

La regla básica es elegir un stack que el equipo conozca bien para minimizar el costo de aprendizaje y el costo de reprogramación. El equipo (3 desarrolladores) domina JavaScript, TypeScript, Python, SQL y NoSQL. Se priorizará la coherencia arquitectónica y la entrega de un producto funcional.

3.2 Stack Tecnológico Seleccionado

- Lenguaje de Cliente (Frontend): Next.js (React) + TypeScript + Tailwind CSS.

Justificación: JavaScript es nativo en navegadores. Next.js permite Server-Side Rendering (SSR), crucial para el SEO de los perfiles de especialistas. Tailwind garantiza un diseño responsive rápido (mobile-first).

- Lenguaje de Servidor (Backend): Node.js + Express + TypeScript.

Justificación: Ideal para sistemas I/O intensivos (múltiples consultas simultáneas a APIs externas). Al usar TypeScript, compartimos tipos e interfaces con el frontend, reduciendo errores de integración.

- Motor de Base de Datos: PostgreSQL (Supabase).

Justificación: Se requiere una estructura de base de datos fuerte y relacional para garantizar la integridad entre Usuarios, Especialistas, Reseñas y Categorías.

- Servicios Externos en la Nube (APIs):
 - YouTube Data API: Para la búsqueda y curación de videos tutoriales (funcionalidad Core).
 - Ollama (3.1:8b): Para el procesamiento de lenguaje natural y diagnóstico (Funcionalidad futura ideal).

- Infraestructura y despliegue: VPS (Ubuntu server) + Docker + Cloudflare + Supabase + Vercel (posible).

Justificación: Docker garantiza que el entorno de desarrollo sea idéntico al de producción. Cloudflare provee SSL automático, CDN y protección DDoS gratuita.

3.3 Arquitectura del Sistema

El sistema seguirá una arquitectura orientada a servicios externos, desplegada en la nube:

1. El usuario accede a www.fixya.com (protegido por Cloudflare Proxy).
2. Cloudflare enruta el tráfico HTTPS al VPS en la nube.
3. En el VPS, Docker orquesta los contenedores: Next.js (Frontend), Node.js (Backend) y la conexión a Supabase (PostgreSQL).
4. Cuando se ejecuta el CU-01, el Backend (Node.js) actúa como orquestador:
 - (Ideal) GET a la API de Ollama para estructurar la consulta.
 - Llama a la API de Youtube para obtener los videos.
 - Consulta a Supabase para obtener los especialistas.
 - Devuelve todo el paquete de datos al Frontend para su renderizado.

4. Propuesta y viabilidad

4.1 Definición formal de la propuesta

"FixYa" es una plataforma web responsive que centraliza la resolución de problemas cotidianos mediante la curación inteligente de videos tutoriales (vía Youtube API) y la conexión con un directorio de especialistas verificados manualmente.

Alcance del MVP (Core garantizado): Búsqueda de videos, registro con Google, directorio de especialistas, sistema de reseñas, panel de admin.

Ampliación (Ideal): Integración de IA (Ollama) para generar diagnósticos personalizados y mejorar la precisión de las búsquedas.

Limitaciones: No procesa pagos (el acuerdo económico es exclusivamente entre las partes). No reemplaza consejo médico, legal o de alta tensión.

4.2 Evaluación de viabilidad

Viabilidad Técnica (alta): El equipo domina el stack seleccionado (TS/Node/Postgres). La dependencia de APIs externas (Youtube) es estándar en la industria y está ampliamente documentada.

Viabilidad Temporal (alta): El plazo es de 2 meses (4 sprints de 15 días). La estrategia de "MVP escalonado" garantiza que, si surge un imprevisto con la

integración de la IA, el producto base (Videos + Especialistas) estará 100% funcional y presentable.

Viabilidad Operativa (alta): La validación manual de especialistas (CU-05) es perfectamente operable para un volumen inicial, sin requerir inversión en APIs de verificación de identidad.

4.3 Análisis FODA

Fortalezas: Stack coherente y conocido por el equipo. Arquitectura en la nube robusta. Estrategia de MVP escalonado que mitiga el riesgo de no llegar con la IA.

Debilidades: Dependencia de APIs de terceros (Youtube y VPS) para las funcionalidades principales.

Oportunidades: Crecimiento del mercado "Hágalo usted mismo". Los especialistas buscan alternativas orgánicas a las redes sociales para conseguir clientes.

Amenazas: Cambios en los términos de servicio, cuotas o costos de las APIs externas. Competencia de marketplaces generalistas.

4.4 Identificación de riesgos y mitigaciones

Para garantizar la viabilidad del proyecto, se identificaron los principales riesgos que podrían afectar el desarrollo, el despliegue o la operación de "FixYa", junto con sus respectivos planes de mitigación.

Riesgo 1: No llegar a integrar la Inteligencia Artificial a tiempo.

Este riesgo tiene un impacto medio sobre el proyecto. La implementación de Ollama en la VPS requiere la instalación y configuración del modelo, ajuste del prompt, y validación de que el hardware de la VPS pueda ejecutar el modelo de forma eficiente. El plan de mitigación consiste en la estrategia de MVP escalonado que se definió en la propuesta. El sistema está diseñado con un patrón de respaldo (fallback): si la IA no está integrada al momento de la entrega, el backend utiliza el texto crudo ingresado por el usuario como palabras clave para consultar directamente la API de Youtube y filtrar los especialistas en la base de datos. De esta manera, el producto base (buscador de videos + directorio de especialistas) se entrega completamente funcional independientemente del estado de la integración de la IA.

Riesgo 2: Fuera del alcance.

Este riesgo tiene un impacto alto. Durante el desarrollo, es probable que surjan ideas de funcionalidades adicionales (por ejemplo, pagos integrados, chat en tiempo

real, app móvil) que, si se incorporan sin control, pueden desviar al equipo del objetivo principal y comprometer la entrega en plazo. El plan de mitigación es la adhesión estricta a la tabla de 6 Casos de Uso definida en la sección 2.2 de este documento. Cualquier funcionalidad que no esté contemplada en esos casos de uso se documenta explícitamente como "futura ampliación" y no se desarrolla durante el MVP. Las decisiones de alcance se revisan en cada retrospectiva de Sprint.

Riesgo 3: La IA genera respuestas inseguras o inadecuadas.

Este riesgo tiene un impacto crítico. Si el asistente de IA proporciona instrucciones incorrectas o peligrosas sobre temas sensibles (manipulación de gas, electricidad de alta tensión, diagnósticos médicos o asesoramiento legal), podría generar daño al usuario y exponer a la empresa a responsabilidad legal. El plan de mitigación opera en tres niveles. Primero, a nivel de prompt: las instrucciones enviadas al modelo local de Ollama incluyen restricciones explícitas que prohíben generar respuestas sobre temas de salud, legales o de alta tensión, y obligan a recomendar contactar a un profesional matriculado en esos casos. Segundo, a nivel de interfaz: se incluye un disclaimer visible y permanente en la sección del chat que indica que la información es orientativa y no reemplaza el consejo profesional. Tercero, a nivel legal: los Términos y Condiciones de la plataforma eximen a la empresa de responsabilidad por el uso que el usuario haga de la información proporcionada por la IA.

Riesgo 4: Responsabilidad civil derivada del trabajo de los especialistas.

Este riesgo tiene un impacto alto. Si un especialista registrado en la plataforma realiza un trabajo deficiente, causa un daño material o comete un fraude contra un usuario, existe el riesgo de que el usuario responsabilice a la plataforma. El plan de mitigación se basa en el blindaje legal de la empresa. Los Términos y Condiciones definen explícitamente a la plataforma como un "mero intermediario" que facilita la conexión entre partes, sin ser parte del contrato de servicio ni garantizar la calidad del trabajo presencial. Además, la plataforma no procesa pagos: el acuerdo económico se realiza exclusivamente entre el usuario y el especialista, lo que refuerza la independencia contractual. La validación manual de identidad (DNI y certificados) actúa como filtro preventivo para reducir la probabilidad de perfiles fraudulentos.

Riesgo 5: Caída o indisponibilidad del servicio en la nube.

Este riesgo tiene un impacto medio. Dado que toda la infraestructura depende de servicios en la nube (VPS, base de datos gestionada, APIs externas), una caída del proveedor podría dejar la plataforma inaccesible. El plan de mitigación incluye tres medidas. Primero, el uso de Docker en el VPS con la política de reinicio automático (restart: always), que garantiza que los contenedores se recuperen solos ante fallos menores. Segundo, la configuración de backups diarios automáticos de la base de

datos Supabase y/o en la VPS, almacenados en un disco separado, para permitir la recuperación de datos en caso de pérdida. Tercero, la protección de Cloudflare como capa de defensa contra ataques DDoS y como CDN que puede servir una página de mantenimiento estática si el servidor de origen no responde.

5. Plan de trabajo (metodología Scrum - 4 Sprints)

5.1 Arquitectura por capas

Antes de detallar los sprints, es fundamental definir cómo se organiza el desarrollo en capas técnicas, ya que cada sprint tendrá entregas paralelas en cada una de ellas:

Frontend (Next.js + TypeScript + Tailwind): Capa de presentación. Responsable de la UI/UX, el renderizado SSR para SEO, y la interacción con el usuario final. Se desplegará en Vercel (o en la VPS).

BFF / Backend (Node.js + Express + TypeScript): Capa de lógica de negocio y orquestación. Actúa como Backend for Frontend (BFF): recibe las peticiones del frontend, se comunica con Supabase (base de datos), con la API de Youtube y con la API de Ollama, y devuelve datos formateados al frontend. Centraliza la seguridad, las reglas de negocio y la integración con servicios externos.

Base de datos (Supabase - PostgreSQL gestionado): Capa de persistencia. Al ser un servicio gestionado en la nube, elimina la carga operativa de mantener un servidor de base de datos. Incluye autenticación nativa, storage para archivos (DNI/certificados) y Row Level Security (RLS).

Servicios externos en la nube: API de Youtube Data(búsqueda de videos) y API de Ollama (IA).

5.2 Detalle de sprints

SPRINT 1 (Días 1-15): Fundación, Autenticación e Infraestructura en la Nube

Objetivo: Tener la arquitectura base funcionando en la nube, con autenticación operativa y base de datos inicial.

Base de Datos (Supabase):

- Crear proyecto en Supabase y configurar el cluster de PostgreSQL en la nube.
- Diseñar y migrar el esquema inicial: tabla Usuario, tabla Categoría (con seed de categorías: Plomería, Electricidad, Educación, Tecnología).
- Configurar Row Level Security (RLS) para proteger las tablas.
- Habilitar Supabase Storage para la subida de imágenes (DNI, certificados).

Backend / BFF (Node.js + Express + TypeScript):

- Inicializar el proyecto con TypeScript.
- Configurar conexión con Supabase mediante el cliente oficial.
- Implementar el flujo de autenticación con Google OAuth (usaremos el provider nativo de Supabase Auth).
- Crear middleware de autenticación que valide el JWT en cada request protegido.
- Endpoint GET /api/auth/v1 para obtener datos del usuario logueado.

Frontend (Next.js + Tailwind):

- Inicializar proyecto Next.js 16 y Tailwind CSS.
- Configurar proveedor de autenticación (contexto global de usuario).
- Desarrollar landing page responsive con llamado a la acción "Ingresar con Google".
- Implementar flujo de login con Google (redirección, callback, manejo de sesión).
- Crear layout principal con header (mostrar usuario logueado / botón de login).

Infraestructura:

- Configurar dominio en Cloudflare con Proxy activado (SSL automático).
- Desplegar el Frontend en Vercel (o probablemente la VPS).
- Desplegar el BFF en VPS con Docker.
- Configurar variables de entorno en ambos servicios (URL de Supabase, claves de API).

Entregable del Sprint: Un usuario puede acceder al dominio, loguearse con Google, y ver una página de bienvenida personalizada. La base de datos en Supabase está operativa y el BFF responde en la nube.

SPRINT 2: Módulo de Especialistas y Panel de Administración

Objetivo: Completar los casos de uso CU-04 (Postular Perfil) y CU-05 (Validar Especialista).

Base de Datos (Supabase):

- Crear tabla Especialista con relaciones a Usuario y Categoria. Campos: estado (Pendiente/Aprobado/Rechazado), zona, descripción, teléfono, URLs de documentos.
- Crear políticas: solo el dueño puede editar su perfil, solo el admin puede cambiar el estado.
- Configurar buckets de Storage con políticas de acceso para las imágenes de DNI y certificados.

Backend / BFF:

- Endpoint POST /api/especialistas/v1 para crear perfil (validación de campos, subida de archivos a Supabase Storage).

- Endpoint GET /api/admin/especialistas/v1/pendientes (protegido por rol admin).
- Endpoint PUT /api/admin/especialistas/v1:id/aprobar y /rechazar.
- Integración con servicio de comunicación transaccional (WhatsApp) para notificar al especialista.

Frontend:

- Formulario de registro de especialista (multi-paso: datos personales, oficio/categoría, zona, subida de fotos de DNI y certificado).
- Panel de administración (/admin/v1) con tabla de especialistas pendientes, previsualización de documentos y botones de aprobación/rechazo.
- Modal para ingresar motivo en caso de rechazo.
- Página de confirmación post-envío.

Transversal:

- Crear usuario admin de prueba en Supabase para validar el panel.
- Testing e2e del flujo: especialista se registra → admin lo aprueba → especialista recibe comunicación.

Entregable del Sprint: Un especialista puede registrarse y subir su documentación. Un administrador puede revisar, aprobar o rechazar perfiles desde un panel protegido. El sistema notifica al especialista el resultado.

SPRINT 3: Listado público, búsqueda y Youtube (Hito: MVP Base)

Objetivo: Completar los casos de uso CU-01 (Consultar Buscador) y CU-03 (Contactar Especialista). Este sprint entrega el MVP Base funcional.

Base de Datos (Supabase):

- Crear vistas o funciones SQL optimizadas para: filtrar especialistas por categoría y zona, ordenar por calificación promedio, y limitar a los 3 mejores.
- Crear función para obtener datos públicos de un especialista (sin exponer información sensible).

Backend / BFF:

- Endpoint GET /api/especialistas/v1 con query params (categoria, zona) y ordenamiento por reputación.
- Endpoint GET /api/especialistas/v1:id para perfil público.
- Endpoint POST /api/especialistas/v1:id/contacto para registrar métrica de contacto.
- Integración con Youtube Data API: Endpoint GET /api/buscar/v1 que recibe la consulta del usuario, llama a Youtube API con las palabras clave, y devuelve los 3 videos más relevantes (título, thumbnail, URL embebida).
- Endpoint GET /api/buscar/completo/v1 que combina: videos de Youtube + especialistas filtrados por categoría detectada (en esta etapa, la categoría se infiere por coincidencia simple de texto).

Frontend:

- Página de búsqueda con input destacado y botón de búsqueda.
- Sección de resultados: 3 cards de videos embebidos de Youtube (con reproductor).
- Sección de resultados: listado de especialistas (cards con foto, nombre, oficio, zona, estrellas).
- Página de perfil público del especialista con botón "Contactar por WhatsApp".
- Filtros laterales (categoría, zona) en el listado de especialistas.

Transversal:

- Testing del flujo completo: usuario busca "pileta tapada" → ve 3 videos de Youtube → ve 3 plomeros con mejor reputación → hace clic en contactar → se abre WhatsApp.

Entregable del Sprint (HITO MVP BASE): El producto core funciona. Un usuario puede buscar un problema, ver videos tutoriales de YouTube y contactar especialistas verificados. Aunque la IA no se integre, el producto es totalmente usable y presentable.

SPRINT 4: Reseñas, IA y deploy final

Objetivo: Completar CU-06 (Calificar Especialista), integrar IA si es posible, y dejar el producto en producción.

Base de Datos (Supabase):

- Crear tabla Reseña con restricción de unicidad (un usuario, una reseña por especialista).
- Crear trigger en PostgreSQL que, a una determinada hora, recalcula automáticamente el promedio y total de reseñas del especialista en la tabla Especialista.

Backend / BFF:

- Endpoint POST /api/especialistas/v1:id/resena (valida que el usuario haya contactado al especialista).
- Endpoint GET /api/especialistas/v1:id/resena.
- Integración de IA local con Ollama (ideal): Modificar el endpoint /api/buscar/completo/v1 para que, antes de buscar en YouTube, envíe la consulta del usuario al modelo local de Ollama instalado en el VPS. La IA devuelve: diagnóstico estructurado, palabras clave optimizadas para Youtube, y categoría detectada con mayor precisión. Si Ollama no responde o el modelo no está disponible, se usa el fallback (texto crudo).

Frontend:

- Formulario de calificación (1-5 estrellas + comentario) visible solo si el usuario contactó al especialista.
- Sección de reseñas en el perfil público del especialista.

- Mejoras de UI/UX: loading states, empty states, mensajes de error amigables.
- Si se integra IA: mostrar el diagnóstico estructurado de la IA por encima de los videos y especialistas.

Infraestructura y cierre:

- Deploy final en producción (Vercel para Front, VPS para BFF, Supabase para BD).
- Configuración de dominio definitivo con Cloudflare (www.fixya.com)
- Redacción y publicación de Términos y Condiciones y Política de Privacidad (blindaje legal).
- Seed de datos reales: cargar un mínimo de 5 especialistas de prueba contactados por el equipo.
- Testing integral (responsive, flujos críticos, performance).
- Preparación de la presentación final.

Entregable del Sprint: Producto completo desplegado en producción, con sistema de reseñas funcionando, IA integrada (si se alcanzó el ideal), y toda la documentación legal y técnica lista para la presentación.

5.3 Resumen visual del plan

Sprint	Foco principal	Casos de Uso	Entregables
Sprint 1	Fundación y Auth	CU-02	Login con Google funcionando + Infra en la nube
Sprint 2	Especialistas y Admin	CU-04, CU-05	Registro y validación de especialistas
Sprint 3	Búsqueda y Youtube	CU-01, CU-03	MVP BASE: Videos + Especialistas
Sprint 4	Reseñas, IA y Deploy	CU-06 + IA (ideal)	Producto final en producción